

Lab 3.5

Task 1: Accessing a notebook instance in Amazon SageMaker

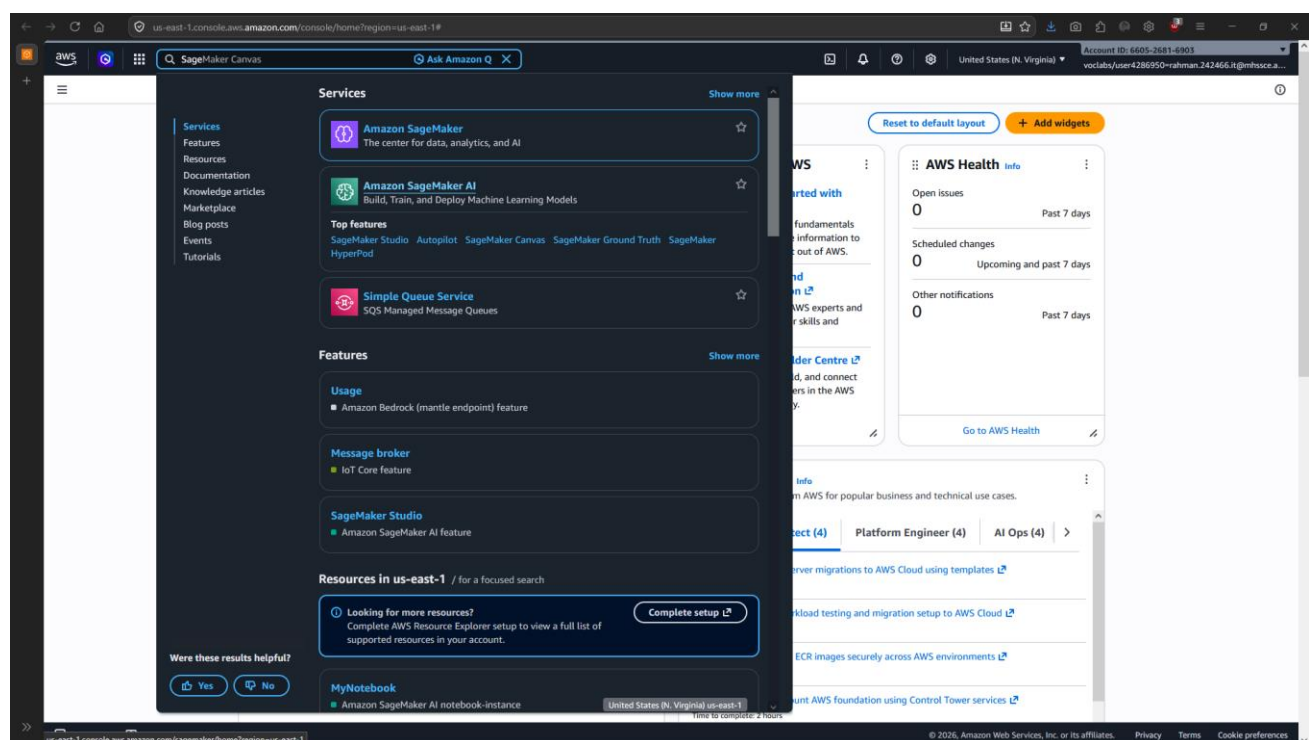
In this task, you will access **Amazon SageMaker** in the AWS Management Console and open the Jupyter notebook for the lab.

To create a Jupyter notebook instance with **Amazon SageMaker**:

On the **AWS Management Console**, on the Services menu, choose **Amazon SageMaker AI**.

From the navigation menu on the left, expand the Applications and IDEs section and choose Notebooks.

Look for the notebook instance named MyNotebook. Go to the end of the row, and open the Jupyter notebook instance by choosing Open JupyterLab.



Task 2: Opening a notebook in your notebook instance

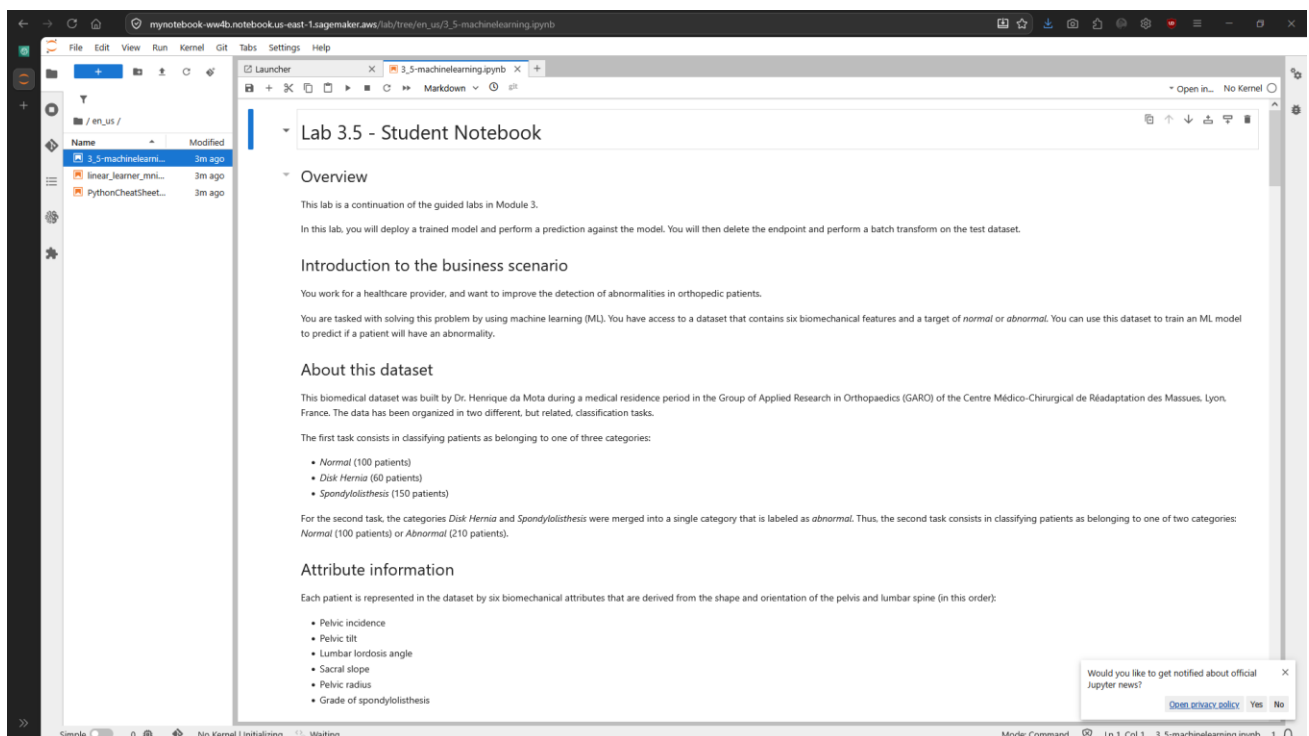
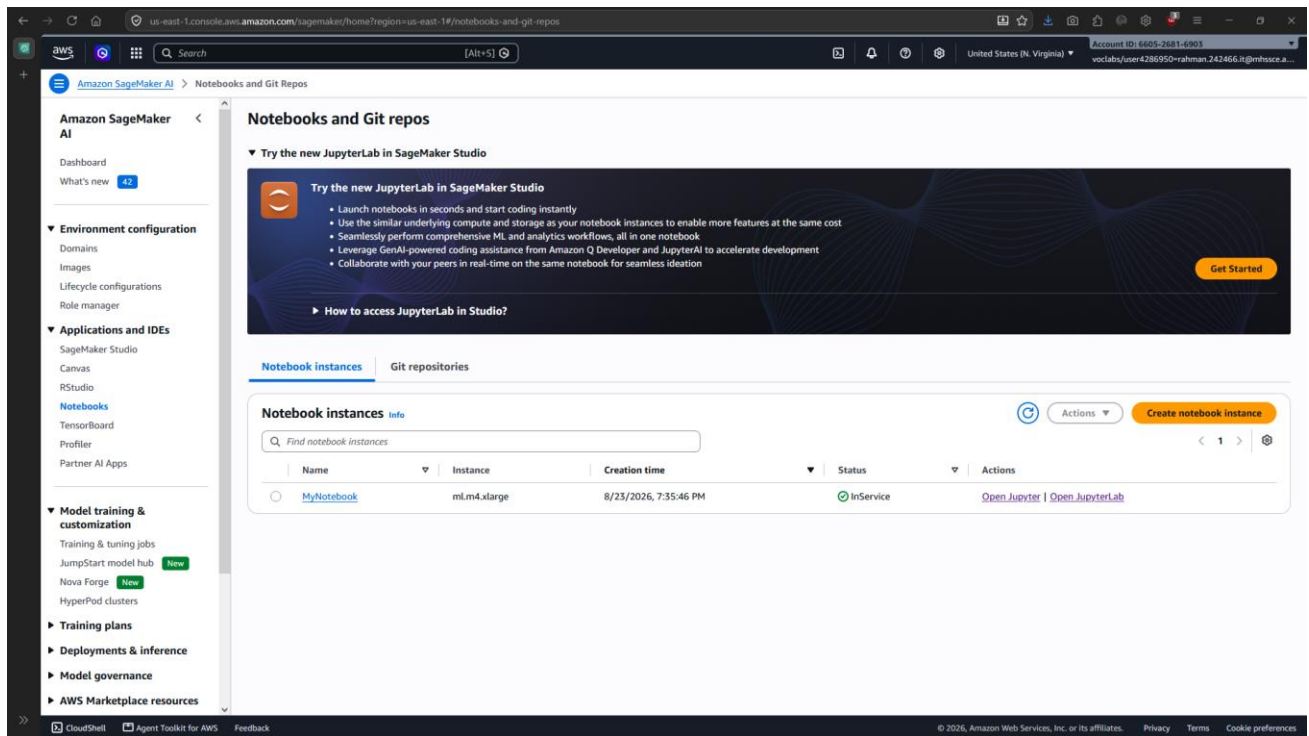
In this task, you will open the notebook for this lab.

In your JupyterLab environment, go to the file browser in the left pane and locate the `3_5-machinelearning.ipynb` file.

Open the `en_us/3_5-machinelearning.ipynb` file by choosing it.

Tip: If a window appears asking you to select a kernel, select `conda_python3` and then choose Select.

Follow the instructions in the notebook.



Notebook Opened

Importing the data

By running the following cells, the data will be imported and ready for use.

Note: The following cells represent the key steps in the previous labs.

```
[1]: bucket='c221098a55751811620812981d660526816901-labbucket-92a061d3ehp'
```

```
[*]: !pip install requests pandas scipy boto3 sagemaker scikit-learn
```

Requirement already satisfied: requests in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (2.34.2)
 Requirement already satisfied: pandas in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (2.3.3)
 Collecting scipy
 Using cached scipy-1.15.3-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
 Requirement already satisfied: boto3 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (1.43.73)
 Requirement already satisfied: sagemaker in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (2.109.0)
 Collecting scikit-learn
 Using cached scikit_learn-1.7.2-cp310-cp310-manylinux2014_x86_64.manylinux2_17_x86_64.whl.metadata (11 kB)
 Requirement already satisfied: charset-normalizer<4,>=2 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from requests) (3.4.9)
 Requirement already satisfied: idna<4,>=2.5 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from requests) (3.10)
 Requirement already satisfied: urllib3<3,>=1.26 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from requests) (2.7.0)
 Requirement already satisfied: certifi<2023.5.7 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from requests) (2026.7.22)
 Requirement already satisfied: numpy<1.22.4 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from pandas) (1.26.4)
 Requirement already satisfied: python-dateutil<2.8.2 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: pytz<2020.1 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from pandas) (2026.3.post1)
 Requirement already satisfied: tzdata>2022.7 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from pandas) (2026.3)
 Requirement already satisfied: botocore<1.44.0,>=1.43.73 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from boto3) (1.43.73)
 Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from boto3) (1.1.0)
 Requirement already satisfied: s3transfer<0.20.0,>=0.19.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from boto3) (0.19.2)
 Requirement already satisfied: six<1.5 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from python-dateutil<2.8.2,2-pandas) (1.17.0)
 Requirement already satisfied: cloudpickle<2.2.1 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (2.2.1)
 Requirement already satisfied: google-pasta in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (0.2.0)
 Requirement already satisfied: protobuf<5.0,>=3.12 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (4.25.9)
 Requirement already satisfied: sagemaker-runtime<1.0.1 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (1.0.1)
 Requirement already satisfied: importlib-metadata<7.0,>=1.4.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (6.11.0)
 Requirement already satisfied: packaging<20.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (24.2)
 Requirement already satisfied: pathos in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (0.3.5)
 Requirement already satisfied: schema in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (0.7.8)
 Requirement already satisfied: PyYAML<6.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (6.0.3)
 Requirement already satisfied: jsonschema in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (4.26.0)
 Requirement already satisfied: platformdirs in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (4.11.0)
 Requirement already satisfied: tblib<1.7.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from sagemaker) (1.7.0)
 Requirement already satisfied: rlp<0.5 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from importlib-metadata<7.0,>=1.4.0-sagemaker) (4.1.0)
 Collecting joblib<1.2.0 (from scikit-learn)
 Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
 Collecting threadpoolctl<=3.1.0 (from scikit-learn)
 Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (11 kB)
 Requirement already satisfied: referencing<0.28.4 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from jsonschema-sagemaker) (0.37.0)
 Requirement already satisfied: rpds-py<0.25.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from jsonschema-sagemaker) (0.30.0)
 Requirement already satisfied: typing-extensions<=4.8.0 in /home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages (from referencing<0.28.4,2-sagemaker) (4.12.0)

Installing Required Dependencies

!pip install requests pandas scipy boto3 sagemaker scikit-learn

Step 1: Hosting the model

Now that you have a trained model, you can host it by using Amazon SageMaker hosting services.

The first step is to deploy the model. Because you have a model object, `xgb_model`, you can use the `deploy` method. For this lab, you will use a single `m4.xlarge` instance.

```
[*]: xgb_predictor = xgb_model.deploy(initial_instance_count=1,
                                     serializable_objects=[xgb_predictor],
                                     instance_type='m4.xlarge')
```

Step 2: Performing predictions

Now that you have a deployed model, you will run some predictions.

First, review the test data and re-familiarize yourself with it.

```
[*]: test.shape
```

You have 31 instances, with seven attributes. The first five instances are:

```
[*]: test.head(5)
```

You don't need to include the target value (class). This predictor can take data in the comma-separated values (CSV) format. You can thus get the first row without the class column by using the following code:

```
test.iloc[:,1:]
```

The `iloc` function takes parameters of [rows,cols]

To only get the first row, use `0:1`. If you want to get row 2, you could use `1:2`.

To get all columns except the first column (col 0), use `1:`

```
[*]: row = test.iloc[0,1:]
row.head()
```

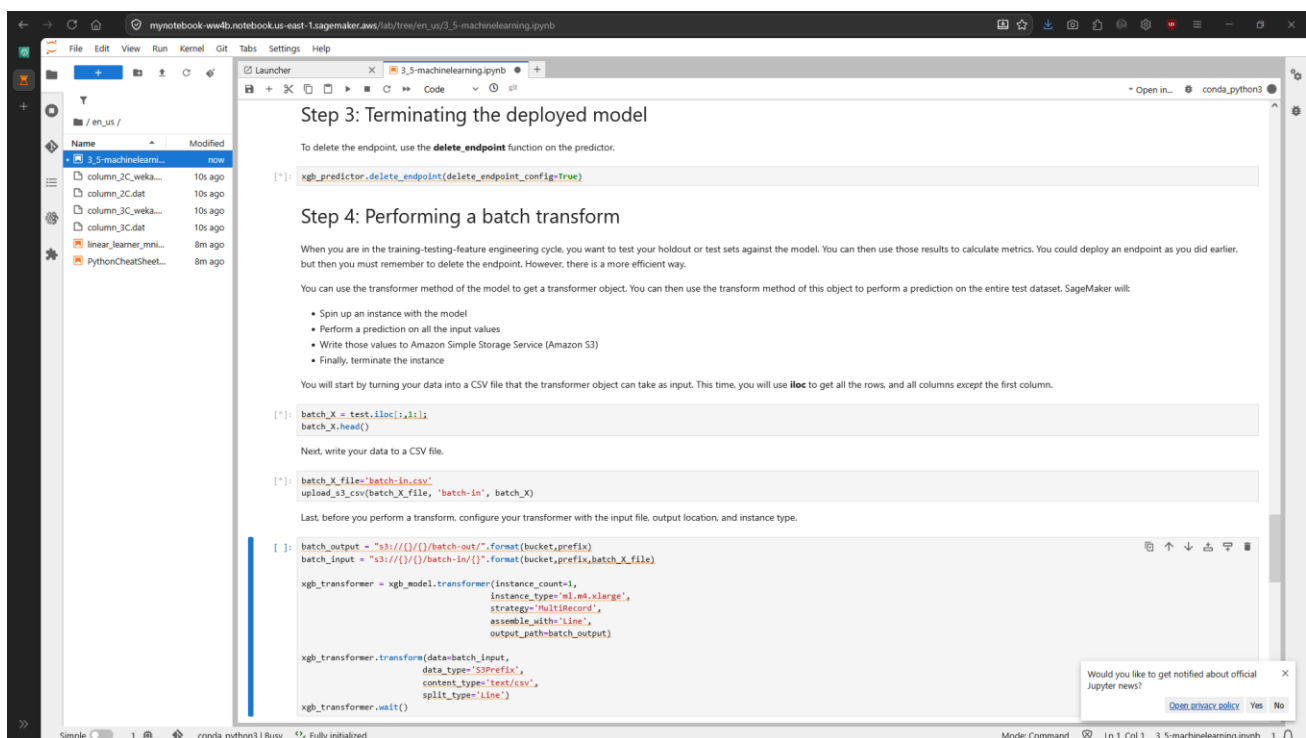
You can convert this to a comma-separated values (CSV) file, and store it in a string buffer.

```
[*]: batch_X_csv_buffer = io.StringIO()
row.to_csv(batch_X_csv_buffer, header=False, index=False)
test_row = batch_X_csv_buffer.getvalue()
print(test_row)
```

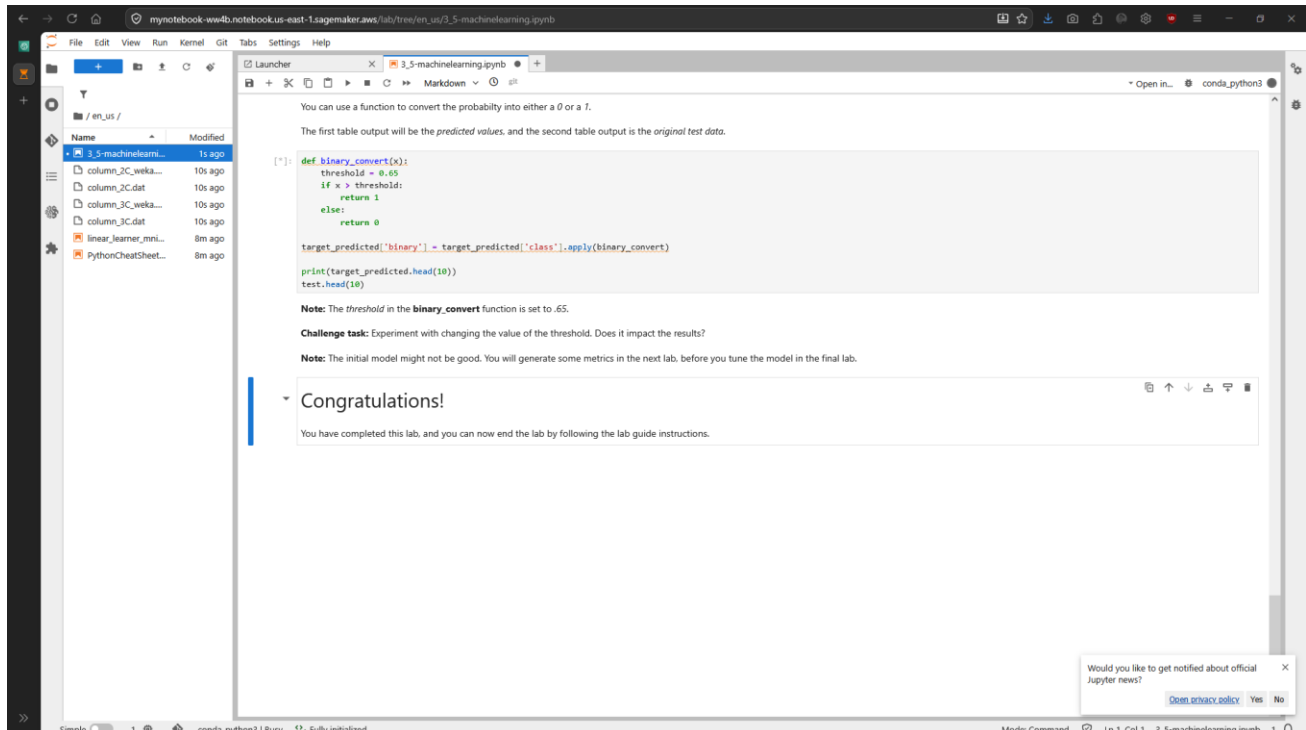
Now, you can use the data to perform a prediction.

```
[*]: xgb_predictor.predict(test_row)
```

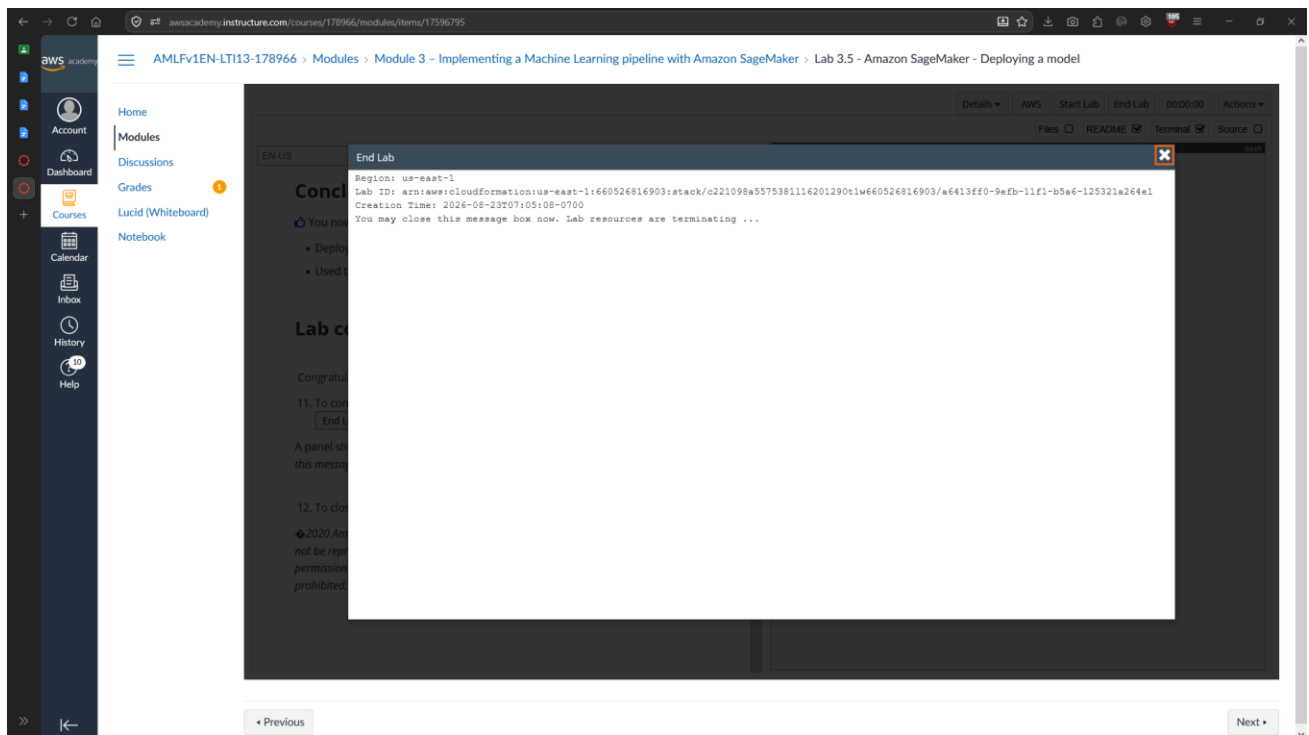
Model deployment endpoint is created successfully using Amazon SageMaker hosting services.



SageMaker endpoint is terminated, followed by configuring batch transform for predictions.



Prediction probabilities are converted into binary classifications using a defined threshold.



Lab completion is confirmed after successfully deploying, testing, and transforming the machine learning model.

Lab 3.6

Task 1: Accessing a notebook instance in Amazon SageMaker

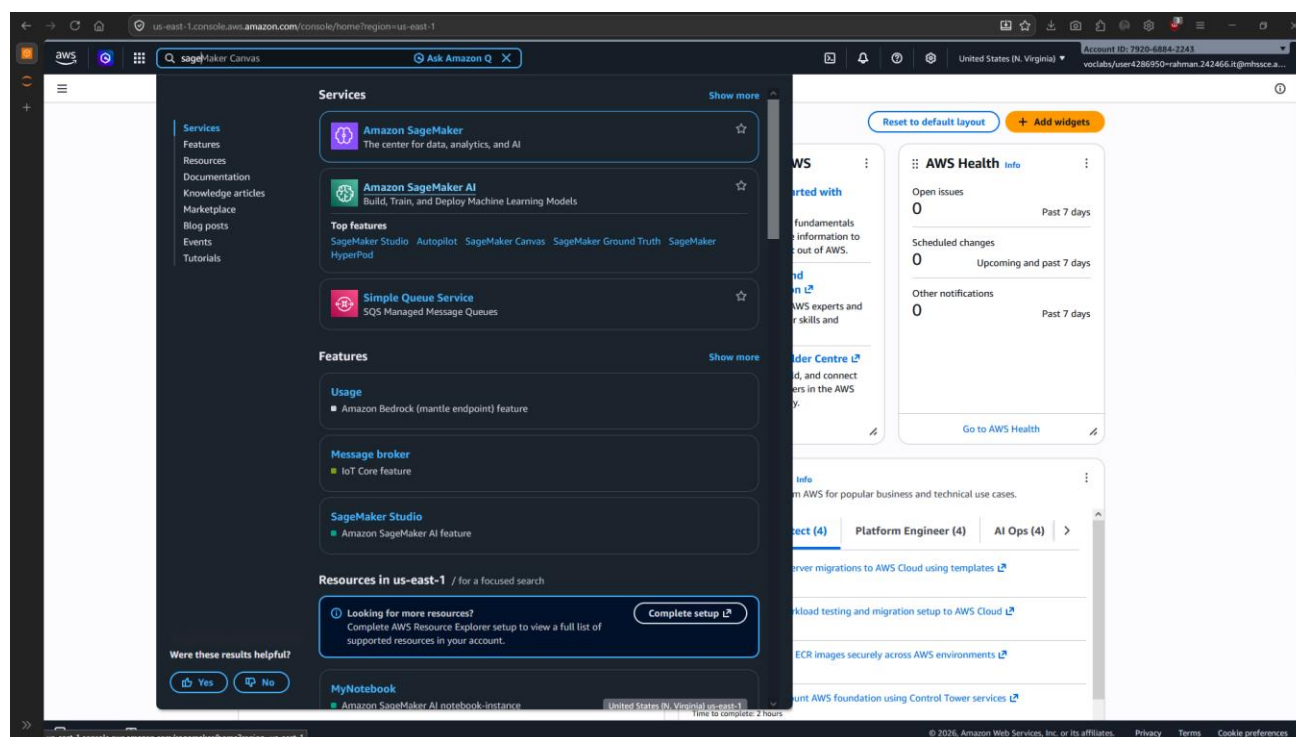
In this task, you will access **Amazon SageMaker** in the AWS Management Console and open the Jupyter notebook for the lab.

To create a Jupyter notebook instance with **Amazon SageMaker**:

On the **AWS Management Console**, on the Services menu, choose **Amazon SageMaker AI**.

From the navigation menu on the left, expand the Applications and IDEs section and choose Notebooks.

Look for the notebook instance named MyNotebook. Go to the end of the row, and open the Jupyter notebook instance by choosing Open JupyterLab.



Task 2: Opening a notebook in your notebook instance

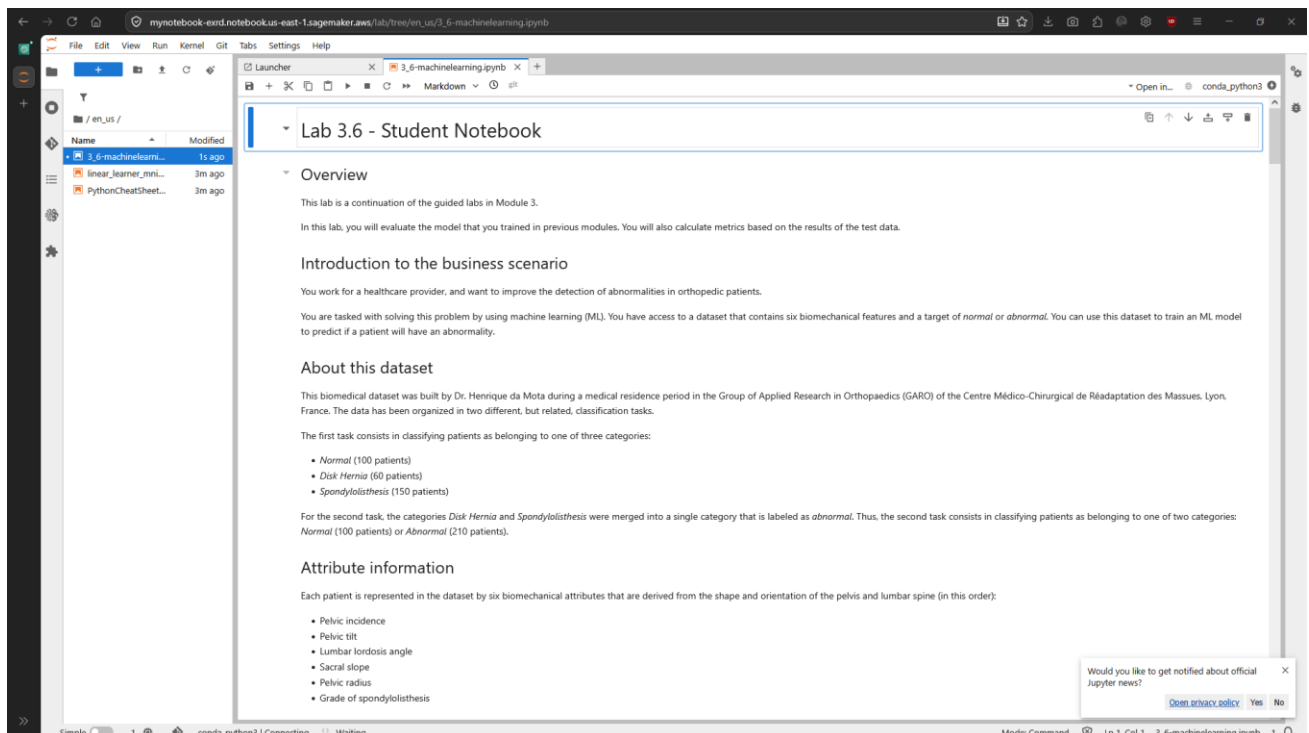
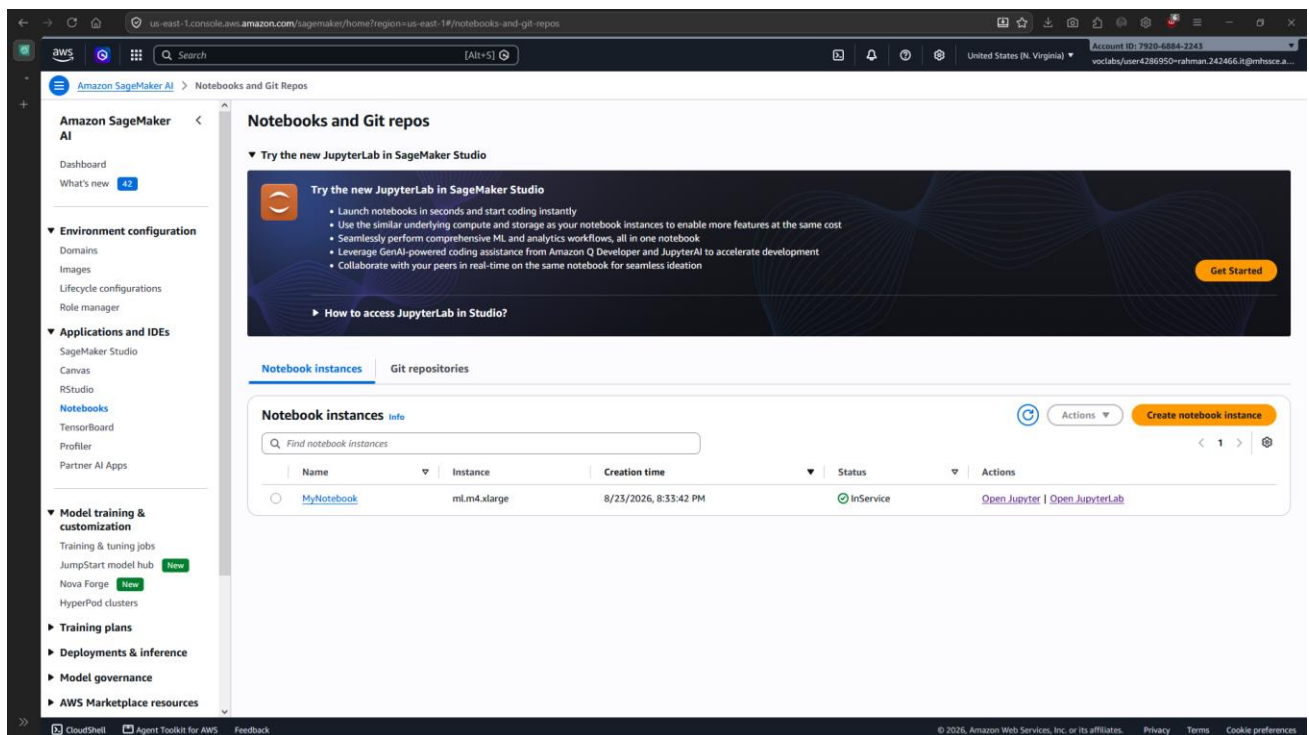
In this task, you will open the notebook for this lab.

In your JupyterLab environment, go to the file browser in the left pane and locate the `3_6-machinelearning.ipynb` file.

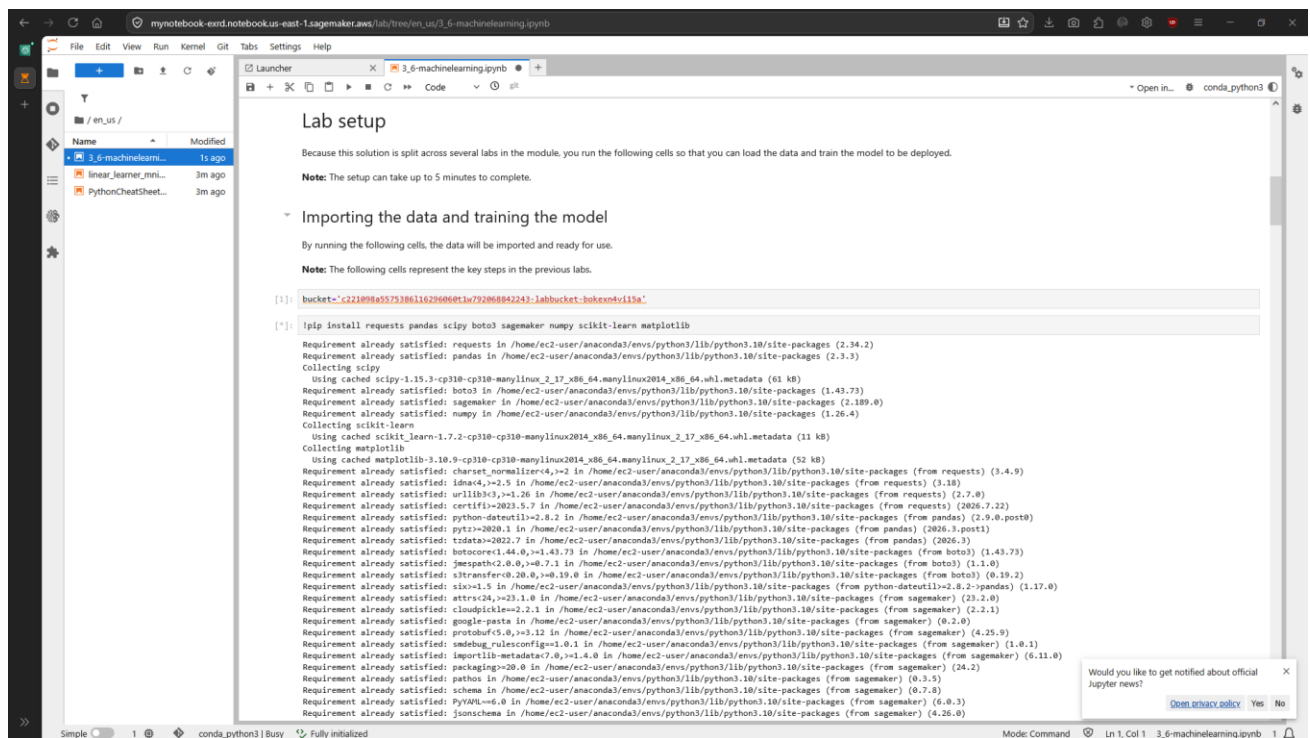
Open the `en_us/3_6-machinelearning.ipynb` file by choosing it.

Tip: If a window appears asking you to select a kernel, select `conda_python3` and then choose Select.

Follow the instructions in the notebook.

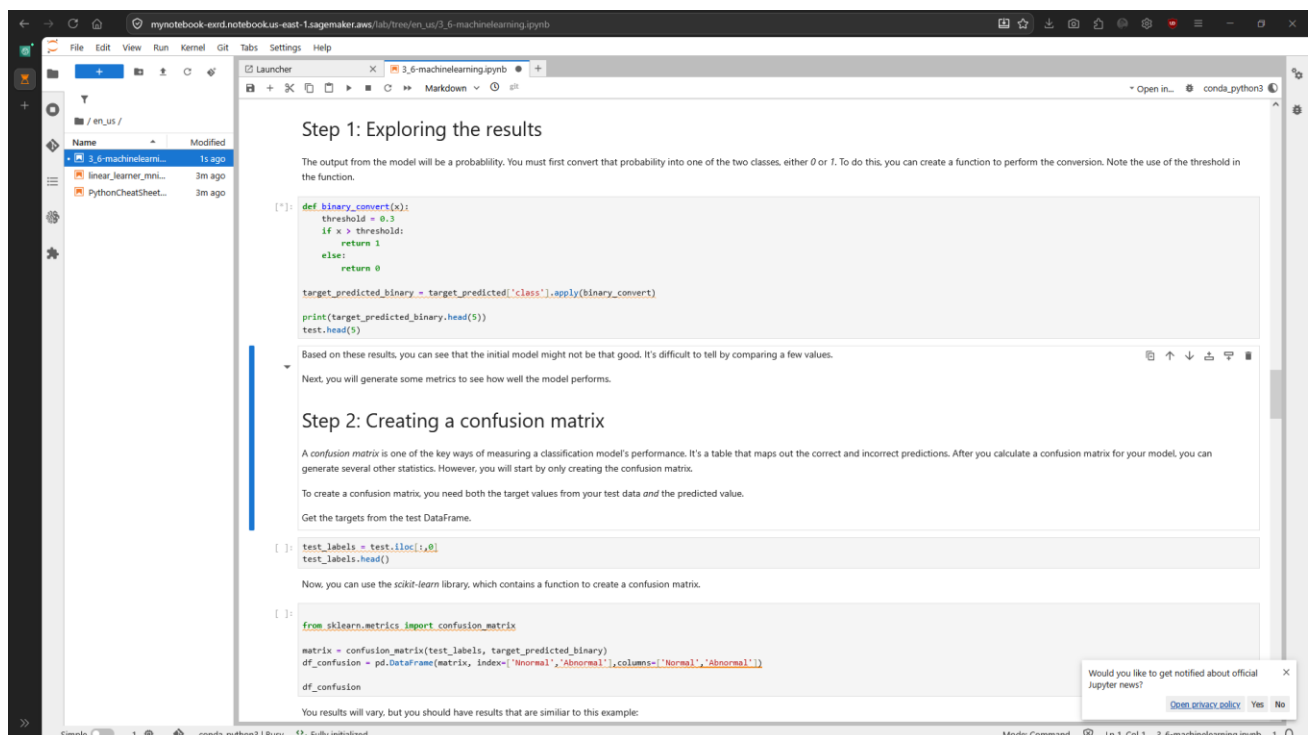


Notebook Opened



Installing Required Dependencies

`!pip install requests pandas scipy boto3 sagemaker numpy scikit-learn matplotlib`



Exploring prediction results and constructing a confusion matrix in Python.

Step 3: Calculating performance statistics

If you want to compare this model to the next model that you create, you need some metrics that you can record. For a binary classification problem, the confusion matrix data can be used to calculate various metrics.

To start, extract the values from the confusion matrix cells into variables.

```
[ ]: from sklearn.metrics import roc_auc_score, roc_curve, auc
TN, FP, FN, TP = confusion_matrix(test_labels, target_predicted_binary).ravel()

print(f"True Negative (TN) : {TN}")
print(f"False Positive (FP): {FP}")
print(f"False Negative (FN): {FN}")
print(f"True Positive (TP) : {TP}")
```

You can now calculate some statistics.

Sensitivity

Sensitivity is also known as *hit rate*, *recall*, or *true positive rate (TPR)*. It measures the proportion of the actual positives that are correctly identified.

In this example, the sensitivity is the probability of detecting an abnormality for patients with an abnormality.

```
[ ]: # Sensitivity, hit rate, recall, or true positive rate
Sensitivity = float(TP)/(TP+FN)*100
print(f"Sensitivity or TPR: {Sensitivity}%")
print(f"There is a {Sensitivity}% chance of detecting patients with an abnormality")
```

Question: Is the sensitivity good enough for this scenario?

Specificity

The next statistic is specificity, which is also known as the *true negative*. It measures the proportion of the actual negatives that are correctly identified.

In this example, the specificity is the probability of detecting normal, for patients who are normal.

```
[ ]: # Specificity or true negative rate
Specificity = float(TN)/(TN+FP)*100
print(f"Specificity or TNR: {Specificity}%")
print(f"There is a {Specificity}% chance of detecting normal patients are normal.")
```

Question: Is this specificity too low, exactly right, or too high? What value would you want to see here, given the scenario?

Extracting prediction metrics to calculate sensitivity and specificity values.

Positive and negative predictive values

The precision, or positive predictive value, is the proportion of positive results.

In this example, the positive predictive value is the probability that subjects with a positive screening test truly have an abnormality.

```
[ ]: # Precision or positive predictive value
Precision = float(TP)/(TP+FP)*100
print(f"Precision: {Precision}%")
print(f"You have an abnormality, and the probability that is correct is {Precision}%")
```

The negative predictive value is the proportion of negative results.

In this example, the negative predictive value is the probability that subjects with a negative screening test truly have an abnormality.

```
[ ]: # Negative predictive value
NPV = float(TN)/(TN+FN)*100
print(f"Negative Predictive Value: {NPV}%")
print(f"You don't have an abnormality, but there is a {NPV}% chance that is incorrect.")
```

Think about the impact of these values. If you were a patient, how worried should you be if the test for an abnormality was positive? On the opposite side, how reassured should you be if you tested negative?

False positive rate

The false positive rate (FPR) is the probability that a false alarm will be raised, or that a positive result will be given when the true value is negative.

```
[ ]: # Fall out or false positive rate
FPR = float(FP)/(FP+TN)*100
print(f"False Positive Rate: {FPR}%")
print(f"There is a {FPR}% chance that this positive result is incorrect.")
```

False negative rate

The false negative rate -- or miss rate -- is the probability that a true positive will be missed by the test.

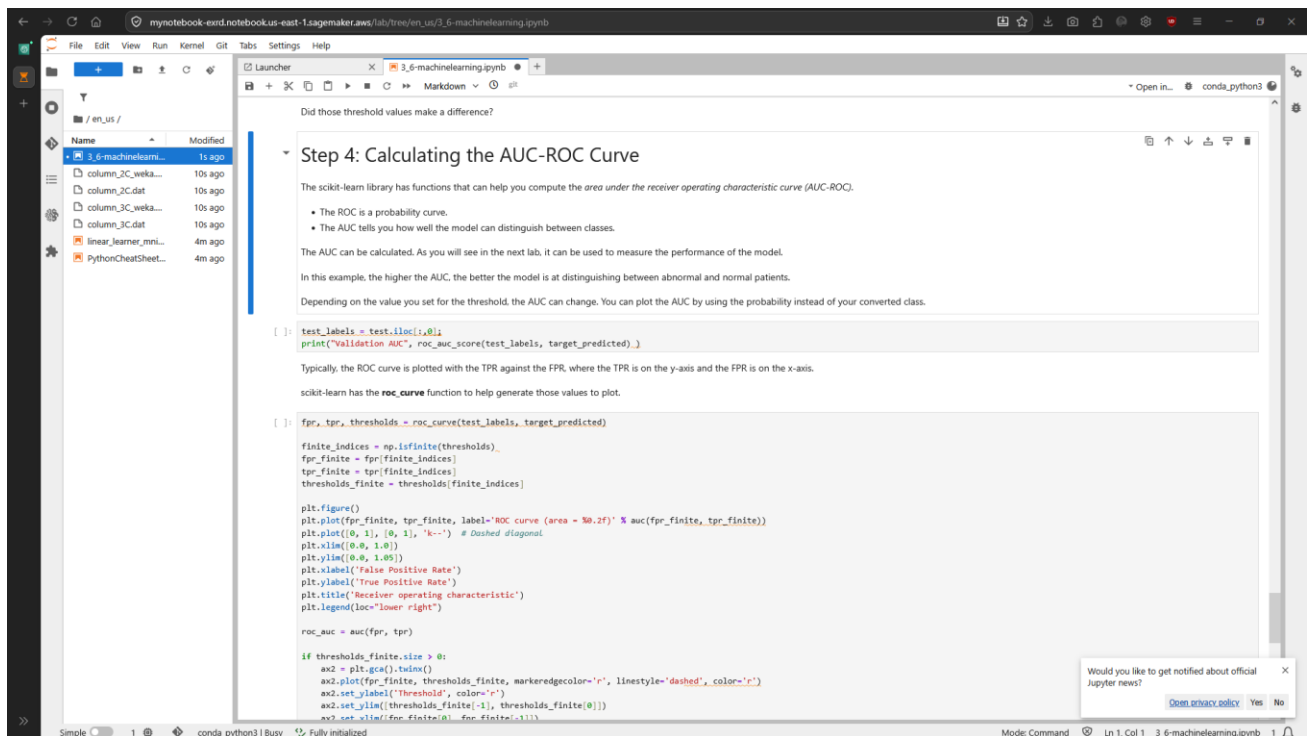
```
[ ]: # False negative rate
FNR = float(FN)/(TP+FN)*100
print(f"False Negative Rate: {FNR}%")
print(f"There is a {FNR}% chance that this negative result is incorrect.")
```

False discovery rate

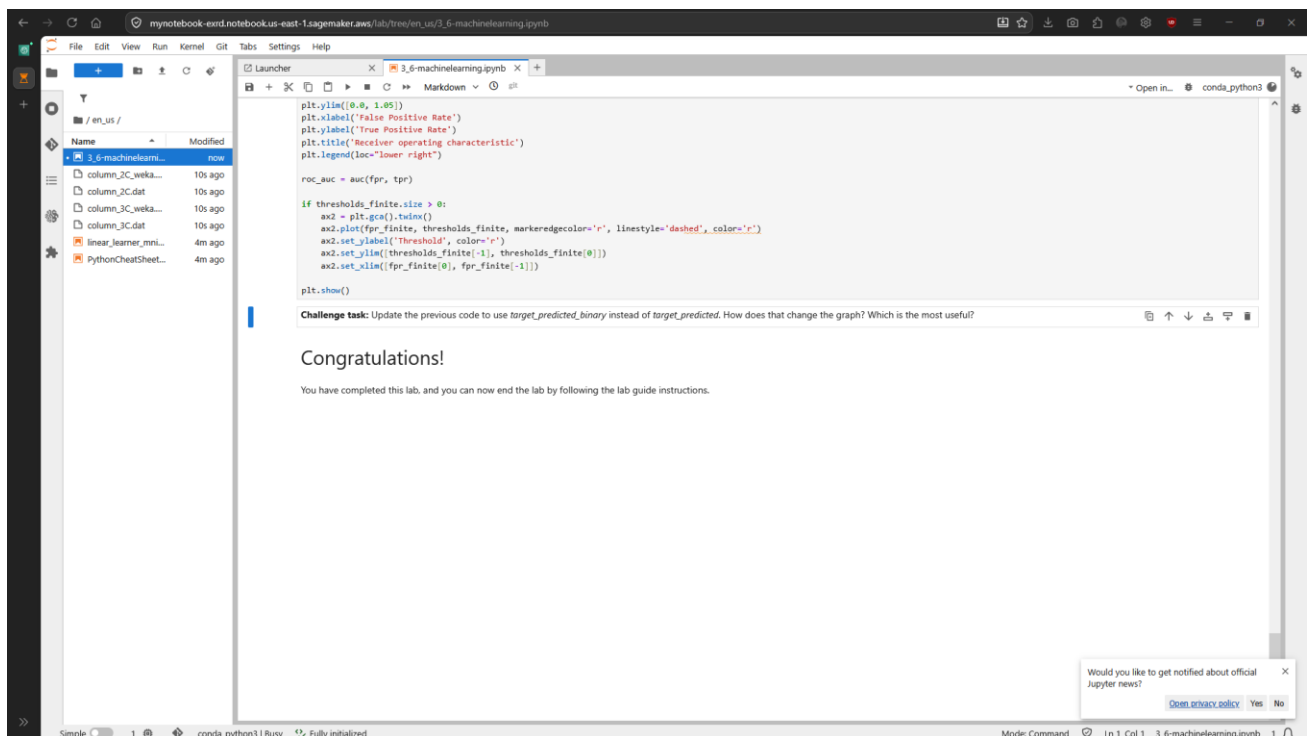
In this example, the false discovery rate is the probability of predicting an abnormality when the patient doesn't have one.

```
[ ]: # False discovery rate
FDR = float(FP)/(TP+FP)*100
```

Coding evaluations for predictive values, false rates, and discovery rates.



Setting up parameters to calculate and plot the AUC-ROC curve.



Finalizing the ROC plot code and completing the machine learning lab.